Rapport de TP : Création d'une API REST

1. Description du projet (Cas d'utilisation)

Le but de ce TP est de développer le backend d'une application de gestion d'étudiants en Python avec Flask et MySQL. Les actions possibles (CRUD) pour l'utilisateur sont :

- Voir la liste de tous les étudiants.
- Voir les détails d'un étudiant en particulier via son ID.
- Ajouter un nouvel étudiant dans la base de données.
- Modifier les informations d'un étudiant existant.
- Supprimer un étudiant.

2. Description de l'API (Swagger)

Méthode	Route HTTP	Action	Codes de retour
GET	/v4/etudiants/	Affiche tous les étudiants	200 (OK), 401 (Refusé)
GET	/v4/etudiants/{id}	Affiche un étudiant spécifique	200 (OK), 404 (Erreur), 401
POST	/v4/etudiants/	Ajoute un étudiant	201 (Créé), 401
PUT	/v4/etudiants/{id}	Modifie un étudiant	200 (OK), 404 (Introuvable), 401
DELETE	/v4/etudiants/{id}	Supprime un étudiant	200 (OK), 404, 401

3. Récapitulatif des versions

- **v1 :** Création basique des routes GET, POST, PUT, DELETE. Connexion simple à la base de données sans gestion des erreurs.
- **v2 :** Ajout de la gestion des erreurs avec des blocs `try / except` (par exemple pour gérer les ID inexistants). Utilisation des f-strings pour les requêtes SQL.
- **v3 :** Ajout d'une sécurité avec l'authentification (login et mot de passe vérifiés dans une table `user`). Seuls les utilisateurs connectés peuvent utiliser l'API.

4. Diagramme de classes

Pour la version 4, le code a été structuré avec la programmation orientée objet. Voici la classe générée par pyreverse :

Database
+ database : str + host : str + password : str + user : str
+ authorized(request) : bool + connect() + create(nom, prenom, email, tel) + delete(id) : int + readAll() : list + readOne(id) : tuple + update(id, nom, prenom, email, tel) : int

5. Fiche de tests avec Postman

Test réalisé	Route testée	Résultat attendu	Résultat obtenu
GET sans mot de passe	/v4/etudiants/	Code 401 (Acces refuse)	OK (401)
GET avec bons identifiants	/v4/etudiants/	Code 200 avec le JSON	OK (200)
GET avec un faux ID (ex: 99)	/v4/etudiants/99	Code 404 (id invalide)	OK (404)
POST pour ajouter	/v4/etudiants/	Code 201 (Ajout OK)	OK (201)

6. Conclusion et Cybersécurité

L'API fonctionne bien pour gérer les étudiants, mais il y a plusieurs problèmes de sécurité à corriger avant de la mettre en ligne :

- **Injections SQL :** Les requêtes SQL utilisent des f-strings (ex: `f"INSERT INTO ... '{data['nom']}'"`). C'est très dangereux car un utilisateur malveillant peut taper du code SQL dans le champ "nom". Il faut utiliser des requêtes préparées avec `%s`.
- **Mots de passe en clair :** Dans la fonction login, on voit que le mot de passe est vérifié en texte clair dans la base de données. Il faudrait stocker les mots de passe hachés.
- **Failles XSS :** Les données entrées ne sont pas vérifiées. Il faudrait s'assurer qu'elles ne contiennent pas de scripts avant de les stocker.